

Laboratório 01

Objetivo: Criação e experimentação sobre uma infraestrutura de máquinas virtuais na plataforma Google Cloud Platform e de *Containers* Docker para alojar e executar serviços ao longo da Unidade Curricular de CD, incluindo os trabalhos de avaliação prática.

Nota: No Anexo 1 estão descritos alguns dos comandos Docker solicitados no guião e comandos para gerir VM no GCP usando a ferramenta de linha de comandos `gcloud`.

1) Seguindo as instruções no ficheiro em anexo, <**CD2526-Registo na Google Cloud Platform como Aluno.pdf**> cada grupo de alunos deve criar um projeto GCP em que todos os alunos do grupo podem gerir recursos nesse projeto (**role owner** do projeto).

2) Seguindo as instruções no ficheiro em anexo, <**CD2526-CriarVms-GCP.pdf**>, crie uma máquina virtual (VM) no serviço *Compute Engine* da GCP. Por questão de uniformização, a VM deve ter o sistema operativo Ubuntu 24.04 LTS (versão x86/64) e a instalação do Java OpenJDK 21.

3) Em anexo encontra o artefacto <**ServiceREST-j21.jar**> que é um serviço HTTP REST que aceita pedidos por omissão no porto 80 ou noutro porto passado como argumento na chamada (`java -jar ServiceREST-j21.jar 7500`). O serviço disponibiliza as seguintes rotas http:

`http://<host ip>:7500/ping`

`http://<host ip>:7500/hello/<some name>`

`http://<host ip>:7500/calc/number{+,-,*}number`

Ex: `http://<host ip>:7500/calc/5*3` retorna 15

a) Teste na sua máquina local (IP localhost) o serviço chamando as rotas num browser;

b) Faça upload do artefacto para a VM criada no ponto 2) e, garanta que a VM tem o porto que quer usar aberto numa *firewall rule*. Execute o serviço HTTP REST e verifique que consegue chamar as várias rotas do serviço a partir de um browser;

c) Verifique que, durante a execução, o serviço gera um ficheiro de *logs* <ServiceRestlogs> na diretoria corrente onde foi executado o serviço. Visualize o conteúdo do ficheiro.

4) Instale o *runtime* Docker na VM de acordo com o site oficial, <https://docs.docker.com/engine/install/ubuntu> (o resumo está no slide 7 de **CD-03 VMs-Containers.pdf**).

5) Verifique que está tudo a funcionar executando e interpretando o seguinte comando:

```
docker run fedora cat /etc/os-release
```

6) Crie o ficheiro *Dockerfile* e com o comando <`docker build`> crie a imagem Docker de um *container* para executar o serviço HTTP REST;

7) Execute o comando `docker images` e verifique que o repositório local da VM contém a imagem que gerou.

8) Execute dois *containers* (docker run) baseados na imagem que gerou e que ficam disponíveis nos portos 8000 e 8001 da VM (*host machine*), garantindo que esses portos estão abertos na *firewall rules* da VM. Verifique que consegue chamar os dois serviços em execução nos dois *containers*.

9) No conteúdo do ficheiro Dockerfile (ver slides da aula) a diretoria WORKDIR (*working directory*) é onde o *container* executa o serviço REST, por exemplo /usr/datafiles. Será aí que é criado o ficheiro de *logging* ServiceRestlogs que regista eventuais erros ou as chamadas às rotas do serviço. No entanto, fora do *container*, e após o *container* terminar, não conseguiremos ler o ficheiro dado que cada *container* tem um *file system* próprio e que não é persistido na máquina *host*.

Crie na máquina *host* uma diretoria (por exemplo; /var/Logs) e ao lançar o container mapeie essa diretoria (*bind mount*) para a diretoria do container onde ficam os *logs*. Verifique que agora já consegue visualizar os *logs* durante a execução ou após o *container* ter terminado, acedendo à diretoria do *host*.

10) Crie um volume docker de nome *logging* e volte a lançar os *containers* mapeando a diretoria de trabalho dentro do container para esse volume. Execute o comando <docker volume ls> para visualizar os volumes existentes;

11) Execute o comando <docker volume inspect logging> para verificar a diretoria do *file system* do *host* onde está o volume. Em princípio será (/var/lib/docker/volumes/logging/_data)

12) Verifique que agora consegue visualizar os *logs* durante a execução ou após o *container* ter terminado, acedendo à diretoria do *host* onde está o volume logging:

```
sudo ls -al /var/lib/docker/volumes/logging/_data
sudo cat /var/lib/docker/volumes/logging/_data/ServiceRestlogs
```

13) Cada grupo de alunos deve criar uma conta no *dockerhub* partilhada para poderem publicar (*push*) as imagens e posteriormente poderem utilizá-las (*pull*) e executar outros *containers* em qualquer máquina com docker.

Autenticação na VM, com a conta do grupo:

```
docker login -u <utilizador-do-grupo>
```

14) Execute os comandos docker que permitem:

- Listar os *containers* em execução <docker ps -a>
- Fazer *start*, *stop*, *kill* e *remove* (rm) de *containers*;
- Listar e remover imagens no repositório local
- Criar uma imagem com *tags* diferentes e no formato exigido para poder realizar *push* de imagens no repositório *dockerhub* <user>/<image name>[:<tag>]
- Após realizar *push* da imagem para o repositório *dockerhub*, apague a imagem no repositório local (comando \$docker image rm <image_name>) e faça *pull* da imagem a partir do *dockerhub*. Execute um novo *container* com essa imagem;
- Outra possibilidade é criar uma segunda VM para verificar que se pode fazer *push* numa VM e fazer *pull* na outra VM para se executarem *containers*. Nos slides (CD2526-CriarVms-GCP.pdf), ilustra-se como se pode, na plataforma GCP, criar novas VMs com a mesma configuração onde se instalou o Docker. Existe em GCP a possibilidade de criar uma <machine image> e depois facilmente criar novas VMs com a mesma configuração.

Anexo 1 – Commandos gcloud alternativos à interface web e comandos Docker

Ferramenta disponível para download e instalação aqui:

<https://docs.cloud.google.com/sdk/docs/install-sdk>

2)

Em alternativa à consola web (a zona pode ser alterada):

```
gcloud services enable compute.googleapis.com
gcloud compute instances create vm-cd \
  --zone=europe-west1-b \
  --machine-type=e2-medium \
  --image-family=ubuntu-2404-lts-amd64 \
  --image-project=ubuntu-os-cloud \
  --boot-disk-size=20GB --tags=cd-lab
# entrar na VM
gcloud compute ssh vm-cd --zone=europe-west1-b
# já dentro da VM: instalar o OpenJDK 21
sudo apt update && sudo apt install -y openjdk-21-jdk
java -version
```

3)

a)

Na sua máquina, num terminal e depois noutro (ou aceda aos endpoints via browser):

```
java -jar ServiceREST-j21.jar 7500

curl http://localhost:7500/ping
curl http://localhost:7500/hello/CD
# as chavetas e o * tem de ir entre aspas na shell
curl "http://localhost:7500/calc/5*3"
```

b)

```
# copiar o artefacto do seu computador para a VM
gcloud compute scp ServiceREST-j21.jar vm-cd:~ --zone=europe-west1-b
# abrir o porto 7500 para as VMs com a etiqueta cd-lab
gcloud compute firewall-rules create cd-allow-7500 \
  --allow=tcp:7500 --direction=INGRESS --target-tags=cd-lab
# descobrir o IP externo da VM
gcloud compute instances describe vm-cd --zone=europe-west1-b \
  --format="get(networkInterfaces[0].accessConfigs[0].natIP)"
# na VM: executar o servico (portos < 1024 exigem sudo)
gcloud compute ssh vm-cd --zone=europe-west1-b
java -jar ServiceREST-j21.jar 7500 &
```

c)

```
ls -l ServiceRestlogs
cat ServiceRestlogs
# acompanhar em tempo real enquanto chama as rotas
tail -f ServiceRestlogs
```

6)

Na diretoria onde está o ficheiro ServiceREST-j21.jar:

```
# conteudo do Dockerfile: ver o slide do Dockerfile do ServiceREST
nano Dockerfile
docker build -t servicerest .
```

8)

```
# executar na VM
docker run -d --name rest1 -p 8000:80 servicerest
docker run -d --name rest2 -p 8001:80 servicerest
docker ps
# abrir os dois portos na firewall (usado gcloud no seu computador)
gcloud compute firewall-rules create cd-allow-8000-8001 \
  --allow=tcp:8000,tcp:8001 --direction=INGRESS \
  --target-tags=cd-lab
# testar a partir do exterior (no seu computador)
curl http://<ip-externo-da-vm>:8000/ping
curl http://<ip-externo-da-vm>:8001/ping
```

9)

```
sudo mkdir -p /var/logs
docker rm -f rest1
docker run -d --name rest1 -p 8000:80 -v /var/logs:/usr/datafiles servicerest
# os logs passam a ser visiveis no host
ls -l /var/logs
cat /var/logs/ServiceRestlogs
```

10)

```
docker volume create logging
docker rm -f rest1 rest2
docker run -d --name rest1 -p 8000:80 -v logging:/usr/datafiles servicerest
docker run -d --name rest2 -p 8001:80 -v logging:/usr/datafiles servicerest
docker volume ls
```

14)

- a) Listar os *containers* em execução <docker ps -a>
- b) Fazer *start*, *stop*, *kill* e *remove* (rm) de *containers*;

```
docker stop rest1
docker start rest1
docker kill rest1
docker rm rest1
# parar e remover numa so operacao
docker rm -f rest2
```

- c) Listar e remover imagens no repositório local

```
docker images
docker image rm servicerest
# remover todas as imagens sem container associado
docker image prune -a
```

- d) Criar uma imagem com *tags* diferentes e no formato exigido para poder realizar *push* de imagens no repositório *dockerhub* <user>/<image name>[:<tag>]

```
docker tag servicerest <utilizador>/servicerest:v1
# ou fazer build ja com o nome final
docker build -t <utilizador>/servicerest:v1 .
docker images
```

- e) Após realizar *push* da imagem para o repositório *dockerhub*, apague a imagem no repositório local (comando `$docker image rm <image_name>`) e faça *pull* da imagem a partir do *dockerhub*. Execute um novo *container* com essa imagem;

```
docker push <utilizador>/servicerest:v1
docker image rm <utilizador>/servicerest:v1
docker pull <utilizador>/servicerest:v1
docker run -d -p 8000:80 <utilizador>/servicerest:v1
```

- f) Outra possibilidade é criar uma segunda VM para verificar que se pode fazer *push* numa VM e fazer *pull* na outra VM para se executarem *containers*. Nos slides (*CD2526-CriarVms-GCP.pdf*), ilustra-se como se pode, na plataforma GCP, criar novas VMs com a mesma configuração onde se instalou o Docker. Existe em GCP a possibilidade de criar uma *<machine image>* e depois facilmente criar novas VMs com a mesma configuração.

```
gcloud compute instances stop vm-cd --zone=europe-west1-b
gcloud compute machine-images create cd-docker-img \
  --source-instance=vm-cd --source-instance-zone=europe-west1-b
gcloud compute instances create vm-cd-2 \
  --source-machine-image=cd-docker-img --zone=europe-west1-b
```